How-To: Permissions, Autopilot, and YOLO Mode

ragent gives an AI agent direct access to your filesystem, shell, network, and version-control systems. The permissions system is the defense-in-depth layer that decides whether each tool call is **allowed**, **denied**, or must **ask** the user for interactive confirmation before it runs.

This guide covers every part of that pipeline:

- The permission rule schema and how rules are evaluated
- File and directory access controls
- The 7-layer bash security model
- Autopilot mode (autonomous, auto-approving operation)
- YOLO mode (bypass all safety checks)
- The interactive permission dialog (TUI and HTTP)
- Hardwired auto-approve rules
- Configuration reference and worked examples

> **Scope:** Permission rules, bash safety, autopilot, YOLO, and the permission dialog. For the full tool catalog see `docs/howtos/tools/INDEX.md`. For custom agent definitions (which carry their own permission rules) see `docs/howtos/custom-agents.md`. For lifecycle hooks that can also veto tool calls see `docs/howtos/hooks.md`. For the TUI walkthrough see `docs/howtos/tutorial.md`.

Table of Contents

- 1. Overview
- 2. Permission Rule Schema
- 3. Permission Categories
- 4. File and Directory Access Control
- 5. The 7-Layer Bash Security Model
- 6. Autopilot Mode
- 7. YOLO Mode
- 8. The Interactive Permission Dialog
- 9. HTTP Server Permission Flow
- 10. Hardwired Auto-Approve Rules
- 11. Configuration Reference
- 12. Custom Agent Permission Rules
- 13. Slash Commands
- 14. Worked Examples

---

1. Overview

Every tool call in ragent passes through a permission check before it executes. The check is performed by `PermissionChecker` (defined in `crates/ragent-config/src/permission.rs` and re-exported via `crates/ragent-agent/src/permission/mod.rs`) and the session processor's `check_permission_with_prompt` helper (defined in `crates/ragent-agent/src/session/permissions.rs`).

The decision flow has five stages, applied in order:

```
Tool Call

1. Hardwired rules        codeindex, team, task, agent, AskUserQuestion, model_info tools auto-ap

2. Auto-approve flags     --yes / auto_approve / YOLO short-circuit to Allow

3. Directory lists       /dirs allowlist/denylist (glob patterns)

4. PermissionChecker       configured ruleset (last-match-wins)

5. Interactive prompt     if the result is "Ask", show the dialog
```

Each stage can short-circuit the pipeline. If any stage returns **Allow** or **Deny**, later stages are skipped. Only when the result is **Ask** does the agent publish a `PermissionRequested` event and wait for the user to respond.

The five-stage pipeline is implemented in `check_permission_with_prompt` (`crates/ragent-agent/src/session/permi`

```rust
pub async fn check_permission_with_prompt(
    checker: &Arc<RwLock<PermissionChecker>>,
    event_bus: &Arc<EventBus>,
    session_id: &str,
    permission: &str,
    resource: &str,
    tool_name: &str,
    auto_approve: bool,
) -> Result<PermissionAction>
```

---

2. Permission Rule Schema

A permission rule maps a **permission category** and a **glob pattern** to an **action**. Rules are defined in the `permissions` array of `ragent.json` (global or project) or in a custom agent's `permissions` block.

Rule Object

```json
{
  "permission": "<category>",
  "pattern": "<glob>",
  "action": "<allow|deny|ask>"
}
```

Field	Type	Description
`permission`	string	Operation category (e.g. `"read"`, `"edit"`, `"bash"`, `"web"`).
`pattern`	string	Glob pattern matched against the resource path. `"*"` matches all.

Field	Type	Description
`action`	string	What to do when the rule matches: `allow`, `deny`, or `ask`.

The Rust struct lives in `crates/ragent-config/src/permission.rs` (re-exported through `crates/ragent-agent/src/permission/mod.rs`):

```rust
pub struct PermissionRule {
    pub permission: Permission,
    pub pattern: Option<String>,
    pub action: PermissionAction,
}

pub enum PermissionAction {
    Allow,
    Deny,
    Ask,
}
```

Evaluation: Last-Match-Wins

Rules are evaluated **top-to-bottom**, and the **last matching rule** wins. This is the opposite of first-match-wins systems and is important for writing rules correctly. Put general rules first and specific overrides last.

```
"permissions": [
  { "permission": "edit", "pattern": "**",       "action": "ask"   },
  { "permission": "edit", "pattern": "src/**",   "action": "allow" },
  { "permission": "edit", "pattern": "secrets/**", "action": "deny" }
]
```

With the above ruleset:

- Editing `README.md` matches the first rule (`**`) → **ask**
- Editing `src/main.rs` matches the first rule (`**`) then the second (`src/**`) → **allow** (last match wins)
- Editing `secrets/api.key` matches all three rules; the last match (`secrets/**`) → **deny**

If **no rule matches**, the default action is **Ask**.

Wildcard Rules

A rule with `permission: "*"` applies to **all** permission categories. These are stored in a separate `wildcard_rules` bucket and evaluated after permission-specific rules.

```
{ "permission": "*", "pattern": "**", "action": "allow" }
```

Always-Grants (Runtime)

When the user presses **a** (allow always) in the permission dialog, the checker records a permanent "always allow" grant via `record_always`. This grant takes precedence over all ruleset entries and persists for the lifetime of the checker (i.e. the session).

```
checker.record_always("edit", "src/**");
// All future edit operations on src/** are auto-approved
```

---

3. Permission Categories

Every tool declares a `permission_category()` string that identifies what kind of operation it performs. The `Permission` enum normalizes both flat names (`"read"`) and namespaced categories (`"file:read"`) to the same variant, so rules keyed on `"read"` match tools that report `"file:read"`.

Standard Categories

Category string	Permission variant	Typical tools
`read`, `file:read`	`Permission::Read`	`read`, `Glob`, `Grep`, `list`, `file_info`, `diff_files`
`Edit`, `file:write`	`Permission::Edit`	`Write`, `create`, `Edit`, `multi_edit`, `append_to_file`, `rm`, `move_file`, `copy_file`, `make_directory`, `patch`, `apply_patch`
`bash`, `bash:execute`	`Permission::Bash`	`Bash`, `bash_reset`, `bg`
`web`	`Permission::Web`	`webfetch`, `websearch`, `mf_fetch`, `mf_search`, `mf_crawl`, `mf_screenshot` (registered only when a drivable headless browser is present; spec `openhands` FR-029)
`network:fetch`	`Permission::Web`	`http_request`
`plan`, `EnterPlanMode`	`Permission::PlanEnter`	`EnterPlanMode`
`task`	`Permission::Task`	`TaskCreate`, `TaskUpdate`, `TaskGet`, `TaskList`
`git:read`	`Permission::Custom`	`git_status`, `git_log`, `git_diff`, `git_show`, `git_branch`
`git:write`	`Permission::Custom`	`git_add`, `git_commit`, `git_push`, `git_checkout`, `git_merge`, `git_reset`, `git_tag`
`github:read`	`Permission::Custom`	`github_list_issues`, `github_get_issue`, `github_list_prs`
`github:write`	`Permission::Custom`	`github_create_issue`, `github_create_pr`, `github_merge_pr`
`gitlab:read`	`Permission::Custom`	`gitlab_list_issues`, `gitlab_list_mrs`
`gitlab:write`	`Permission::Custom`	`gitlab_create_mr`, `gitlab_merge`
`codeindex:read`	`Permission::Custom`	`codeindex_search`, `codeindex_symbols`, `codeindex_references`
`codeindex:write`	`Permission::Custom`	`codeindex_reindex`
`team:manage`	`Permission::Custom`	`team_create`, `team_spawn`, `team_cleanup`, `team_task_cancel`, `team_assign_task`
`team:communicate`	`Permission::Custom`	`team_message`, `team_broadcast`, `team_read_messages`
`agent:spawn`	`Permission::Custom`	`new_agent`
`agent:control`	`Permission::Custom`	`cancel_agent`, `wait_agents`

Category string	Permission variant	Typical tools
`agent:read`	`Permission::Custom`	`list_agents`
`mcp`	`Permission::Custom`	`mcp_tool`
`none`	(no permission needed)	`think`, `agent_complete`

The `Permission` enum is defined in `crates/ragent-config/src/permission.rs` (re-exported through `crates/ragent-agent/src/permission/mod.rs`):

```
pub enum Permission {
    Read,
    Edit,
    Bash,
    Web,
    Question,
    PlanEnter,
    PlanExit,
    Task,
    ExternalDirectory,
    DoomLoop,
    Custom(String),
}
```

The `From<&str>` implementation normalizes namespaced categories. For example, `"file:write"` strips the namespace prefix and maps to `Permission::Edit`, while `"git:write"` (which has no standard variant) becomes `Permission::Custom("git:write")`. Both the exact custom string and the normalized form are checked during rule evaluation, so a rule keyed on `Permission::Edit` matches a tool reporting `"file:write"`.

A category is permission vocabulary, not a tool name

A category is derived from *what a tool does*, independently of the name that tool is registered under (FR-028). The rename (spec `uniformtools` FR-023) moved thirteen tool *names*; it moved no category, so a rule you wrote against one keeps governing the tool:

```
{ "permission": "bash", "action": "allow", "pattern": "git status*" }
```

still allows the tool now called `Bash`, and

```
{ "permission": "ask_user", "action": "allow" }
```

still allows the tool now called `AskUserQuestion`.

Both of those read like a missed migration and are the requirement. The rename took the tool names to Claude Code's spellings; the families `bash` and `ask_user` are ragent's own permission vocabulary and are unchanged, which is why `permission_category()` still reports `bash:execute` for `Bash` and `ask_user` for `AskUserQuestion`. `specs/uniformtools/T-041-BASELINE.md` records 32 distinct categories across the registry, before and after the rename; spec `agentssetup` FR-005 later grew that to 33 by giving `list_agents` the `agent:read` category it previously lacked (it reported `none`, which is what a tool with no category reports - see the table above).

Default Permission Ruleset

Built-in agents ship with a default ruleset (defined in `crates/ragent-agent/src/agent/mod.rs`):

```
pub fn default_permissions() -> PermissionRuleset {
    vec![
        rule(Permission::Read, "**", PermissionAction::Allow),
        rule(Permission::Edit, "**", PermissionAction::Ask),
        rule(Permission::Bash, "*", PermissionAction::Ask),
        rule(Permission::Web, "*", PermissionAction::Ask),
        rule(Permission::PlanEnter, "*", PermissionAction::Ask),
        rule(Permission::Task, "*", PermissionAction::Allow),
        // Auto-approve the six codeindex read/index tools...
        rule(Permission::Custom("tool:codeindex_search".into()), "*", PermissionAction::Allow),
        rule(Permission::Custom("tool:codeindex_symbols".into()), "*", PermissionAction::Allow),
        // ... (codeindex_references, _dependencies, _status, _reindex)
        // ... plus model_info
        rule(Permission::Custom("tool:model_info".into()), "*", PermissionAction::Allow),
    ]
}
```

Read-only agents (like the `ask` preset) use a stricter ruleset:

```
fn read_only_permissions() -> PermissionRuleset {
    vec![
        rule(Permission::Read, "**", PermissionAction::Allow),
        rule(Permission::Edit, "**", PermissionAction::Deny),
        rule(Permission::Bash, "*", PermissionAction::Deny),
    ]
}
```

---

4. File and Directory Access Control

File operations get extra scrutiny because they can modify or delete your work. Three independent mechanisms guard file access.

4.1 Working-Directory Auto-Read

Read operations (`file:read`, `read`) on paths **inside the current working directory** are auto-approved without any ruleset check. This is implemented in `check_permission_with_prompt`:

```
if permission == "file:read" || permission == "read" {
    if let Ok(cwd) = std::env::current_dir() {
        if let Ok(resource_path) = std::path::Path::new(resource).canonicalize() {
            if resource_path.starts_with(&cwd) {
                return Ok(PermissionAction::Allow);
            }
        } else if !resource.starts_with('/') && !resource.starts_with("..") {
            // Relative path within project, not yet created
            return Ok(PermissionAction::Allow);
```

```
        }
    }
}
```

This means the agent can freely read any file under the project root without prompting. Reads of absolute paths outside the project (e.g. `/etc/passwd`) or relative paths starting with `..` fall through to the ruleset.

4.2 Directory Allowlist / Denylist (`/dirs`)

The `dirs` config block provides glob-pattern allow/deny lists for file operations. These are checked **before** the permission ruleset and take precedence over it. Denylist entries always win over allowlist entries.

Defined in `crates/ragent-config/src/dir_lists.rs` and configured in `ragent.json`:

```
{
  "dirs": {
    "allowlist": ["src/**/*.rs", "docs/**/*.md"],
    "denylist": ["secrets/**", "*.env", ".ssh/**"]
  }
}
```

List	Match type	Effect
`allowlist`	glob	File ops matching → auto-allow (no prompt)
`denylist`	glob	File ops matching → auto-deny (no prompt)

Precedence: denylist > allowlist > working-directory auto-read > ruleset.

4.3 Built-in Denylist

A built-in denylist blocks access to system-critical directories on all platforms. These patterns are always active and cannot be overridden by user config (they are checked first):

```
pub const BUILTIN_DENYLIST: &[&str] = &[
    // Linux system directories
    "/bin/**", "/sbin/**", "/boot/**", "/dev/**",
    "/proc/**", "/sys/**", "/etc/**",
    "/usr/bin/**", "/usr/sbin/**", "/usr/lib/**",
    "/lib/**", "/lib64/**",
    // macOS system directories
    "/System/**", "/Library/**", "/Applications/**", "/private/**",
    // Windows system directories
    "C:/Windows/**", "C:/Program Files/**", "C:/Program Files (x86)/**",
];
```

The built-in allowlist is currently empty by design — it can be extended with commonly safe patterns like `target/**` or `.git/**` in future releases.

4.4 Directory Escape Prevention (Bash Layer 3)

Even if a bash command passes all other checks, it is rejected if it attempts to `cd` or `pushd` outside the working directory. See Section 5 for details.

4.5 Path Normalization

Path normalization resolves `..` segments and symlinks before permission checks. The `check_permission_with_prompt` function calls `Path::canonicalize()` on the resource path and compares it against the canonicalized working directory. This prevents tricks like `cd ../etc` or symlink redirection from bypassing the directory guard.

---

5. The 7-Layer Bash Security Model

Before a shell command reaches the permission system, the `BashTool` (`crates/ragent-tools-core/src/bash.rs`) validates it through seven layers in this order; any rejection stops execution. The layer-by-layer reference (`SAFE_COMMANDS`, `BANNED_COMMANDS`, the denied lists, examples) lives in bash.md - The seven-layer security model.

Order	Layer	Bypassed by YOLO	Bypassed by allowlist
1	Safe-command whitelist	-	-
2	Banned commands	yes	**yes**
3	Directory-escape prevention	no	no
4	Syntax validation	no	no
5	Denied commands & patterns	yes	no
6	User denylist	yes	no
7	Obfuscation detection	yes	no

How the layers meet the permission system:

- **Layer 1 only skips the prompt.** A whitelisted command (26 entries, prefix matched, never one containing a shell separator or redirection) is auto-approved at the permission check but still runs every later layer. `git`, `cargo`, `npm`, interpreters and `rm` are not on the list, so they always reach the permission check.
- **The allowlist only exempts Layer 2.** `/bash add allow curl` (or a `bash.allowlist` entry) re-enables a banned tool without YOLO mode; it does not exempt the command from Layers 5-7.
- **The denylist adds to Layer 5.** `/bash add deny <pattern>` (or a `bash.denylist` entry) rejects any command containing the pattern.

- `/bash` changes persist immediately to `.ragent/ragent.json` (project) or `~/.config/ragent/ragent.json` (with `--global`); see bash.md - The `/bash` slash commands.
- After all seven layers pass, the command goes to the permission check (`PermissionChecker` plus the interactive prompt) like any other tool call.

YOLO Interaction with Bash Layers

When YOLO mode is enabled, Layers 2, 5, 6 and 7 are **bypassed** (the command is logged as a warning but allowed to run). Layer 1 (safe whitelist), Layer 3 (directory escape) and Layer 4 (syntax validation) remain active even in YOLO mode. See Section 7. —

6. Autopilot Mode

Autopilot mode lets the agent run autonomously without requiring user approval for each step. It is designed for long-running, low-risk tasks where you want to step away and let the agent work.

6.1 Enabling Autopilot

Use the `/autopilot` slash command in the TUI:

```
/autopilot on
```

Optional flags limit resource consumption:

Flag	Description
`--max-tokens N`	Stop after consuming N tokens total
`--max-time N`	Stop after N seconds of wall-clock time

```
/autopilot on --max-tokens 50000
/autopilot on --max-time 600
/autopilot on --max-tokens 100000 --max-time 1800
```

Check the current state:

```
/autopilot status
```

Disable:

```
/autopilot off
```

6.2 How Autopilot Works

When autopilot is enabled, the TUI state machine (defined in `crates/ragent-tui/src/app/state.rs` and `event_handler.rs`) automatically sends a continuation prompt after each agent turn ends without the agent calling `agent_complete`:

```
// After the agent finishes a turn without calling agent_complete:
if self.autopilot_enabled && *reason != FinishReason::Cancelled {
    // Check time limit
    let time_exceeded = self.autopilot_time_limit_secs
        .and_then(|limit| {
            self.autopilot_started_at
```

```
                .map(|s| s.elapsed().as_secs() >= limit)
        })
        .unwrap_or(false);

    if time_exceeded {
        // Stop autopilot
    } else {
        // Schedule a continuation on the next render tick
        self.autopilot_pending_continue = Some(
            "Continue working on the task. When fully done, call agent_complete with a summary.".
        );
    }
}
```

The continuation prompt is dispatched on the next render tick via `poll_autopilot_continue()`, which checks that the agent is not already processing and that no `TaskCompleted` event was received.

6.3 Autopilot and Permissions

Autopilot does **not** bypass the permission system by itself. It relies on the existing permission rules to decide whether each tool call is allowed. If a tool call requires interactive confirmation (action = `Ask`), the permission dialog still appears and autopilot pauses until the user responds.

To run fully autonomously without any prompts, combine autopilot with one of:

- **`--yes` CLI flag** — auto-approves all permissions without checking rules or prompting (sets `auto_approve = true` on the session processor)
- **YOLO mode** — bypasses the bash banned/denied/obfuscation checks and the user bash denylist (see Section 7); directory-escape and syntax validation still apply
- **Permissive ruleset** — configure `allow` rules for the categories the agent needs (e.g. `"edit": "allow"` for `src/**`)

The SPEC.md describes the interaction:

> Permissions: auto-approve — Only within allowed rule set

6.4 Autopilot Completion

Autopilot ends when any of the following occur:

Trigger	Effect
Agent calls `agent_complete`	Autopilot stops, status bar shows “task complete”
`--max-tokens` budget exceeded	Autopilot stops with “token limit reached”
`--max-time` limit exceeded	Autopilot stops with “time limit reached”
User runs `/autopilot off`	Autopilot stops, returns to interactive mode
User presses `Esc`	Cancels the running agent and autopilot

When the agent signals completion via `agent_complete`, the TUI receives a `TaskCompleted` event and suppresses any pending continuation:

```
// Exit autopilot mode on task completion
if self.autopilot_enabled {
    self.autopilot_enabled = false;
    self.autopilot_started_at = None;
    self.autopilot_pending_continue = None;
    self.status = "task complete".to_string();
}
```

6.5 Status Bar Indicator

The status bar shows the autopilot state as `AutoPilot` with a green (enabled) or red (disabled) indicator:

`AutoPilot:`

See `docs/howtos/tutorial.md` for the full status bar layout.

---

7. YOLO Mode

YOLO mode is the nuclear option: it bypasses **all** command validation and tool restrictions. It is intended for trusted local environments where you control the agent and its inputs completely.

7.1 What YOLO Bypasses

When YOLO mode is enabled (defined in `crates/ragent-config/src/yolo.rs`), the following safety checks are skipped:

Check	Bypassed?	Notes
Bash banned commands (Layer 2)	Yes	curl, wget, nc, nmap, etc. allowed
Bash denied commands (Layer 5)	Yes	mkfs, insmod, useradd, etc. allowed
Bash denied patterns (Layer 5)	Yes	rm -rf /, sudo, dd if=, etc. allowed
User bash denylist (Layer 6)	Yes	User-defined deny patterns ignored
Obfuscation detection (Layer 7)	Yes	base64|bash, eval(),`\xNN` allowed
Interactive permission prompts	Yes	All tools auto-approve
Safe command whitelist (Layer 1)	No	Still checked (but has no blocking effect)
Directory escape check (Layer 3)	No	Still enforced
Syntax validation (Layer 4)	No	Still enforced

The bypass is implemented as early returns in the Bash tool and `check_permission_with_prompt`:

```
// In BashTool::execute:
if contains_banned_command(command) {
    if ragent_config::yolo::is_enabled() {
        tracing::warn!("YOLO mode: allowing banned command tool");
    } else if ragent_config::bash_lists::is_allowlisted(command) {
        tracing::info!("Banned command allowed by user allowlist");
    } else {
        bail!("Command rejected: uses banned external tool ...");
    }
```

```
}

// In check_permission_with_prompt:
if ragent_config::yolo::is_enabled() {
    return Ok(PermissionAction::Allow);
}
```

7.2 Enabling YOLO Mode

There are two ways to enable YOLO mode:

1. Slash command (TUI):

`/yolo`

2. Keyboard shortcut (TUI):

`Alt+Y`

2. CLI flag (permission auto-approve only):

```
ragent --yes
# or the alias:
ragent --no-prompt
```

> **Note:** `--yes` is *not* YOLO mode. It sets `auto_approve = true` on the session processor, which short-circuits `check_permission_with_prompt` before any rules are checked — but it does **not** set the YOLO flag, so the bash security layers (banned commands, denied patterns, obfuscation detection, user denylist) remain enforced. It does not persist to config — it only lasts for the current session.

The `/yolo` slash command and `Alt+Y` shortcut toggle the persistent YOLO flag, which is saved to the **user-global** `ragent.json` (`~/.config/ragent/ragent.json`) and restored on the next startup.

7.3 Automatic sub-agent scope

There is a **second, narrower** way to get the prompt bypass, and it needs no flag: an unattended sub-agent run (`agent.mode == Subagent` - `new_agent` foreground, background or detached, `/spawn`, cron, server) whose tool can *write* skips the interactive prompt, whatever the global `/yolo` flag says.

A sub-agent has no attached user, so a prompt is not a safeguard there - the call would wait out the watchdog and the run would be lost. The scope is a per-turn function of the run's own offered tool surface:

- `subagent_yolo_bypasses_prompt(agent_is_subagent, write_capable_tools, tool)` decides the call; `write_capable_tools` is built once per turn by `SessionProcessor::write_capable_tools` from the run's model-facing definitions.
- A tool is write-capable unless its `permission_category()` is in `READ_ONLY_PERMISSION_CATEGORIES` (`file:read`, `codeindex`, `web`, `web:read`, `network`, `model`, `model:read`, `shell:open`, `memory:read`, `spec:read`, `skill:read`, `git:read`, `github:read`, `gitlab:read`, `cron:read`, `agent:read`, `team:read`, `task`, `system`). An uncategorised (`none`) tool is write-capable, because it bypasses the permission layer entirely. The failure mode of a missing entry is silent and unsafe: a forgotten read category is treated as write-capable, so a pure inspection run is handed the YOLO *write* scope instead of the ordinary prompt policy. `git:read`, `github:read`, `gitlab:read`, `cron:read`, `agent:read`, `team:read`, `model:read`, `web:read`, `task` and `system` were all

missing once and are now listed. `tests/test_read_only_permission_categories.rs` keeps the list honest from both directions: every registry tool's category must be a known category (a typo such as `git:reed` fails loudly), the named read surfaces must classify read-only, and the named write surfaces must still classify write-capable. `team:tasks` stays write-capable because `team_task_claim` and `team_task_complete` share it with the read-only `team_task_list`.

- **Read-only runs get nothing**: an `explore` sub-agent, a plan reviewer, or a loop restricted to reads resolves to an empty set and keeps the ordinary prompt policy.
- **Only the prompt is skipped.** The loop tool-set restriction, the path guards, the 7-layer bash `is_safe_command` stage (which runs *before* the prompt), the tool-repeat guard, every pre-tool hook and an explicit policy `Deny` still apply, and a forced destructive-action checkpoint still prompts (FR-024).
- The **process-wide** parts of YOLO in the table above (banned commands, denied patterns, obfuscation detection, the dynamic-context allowlist and MCP config validation) are still governed by the single global flag: a sub-agent run does not enable them on its own.

7.4 Persistence

YOLO is a user-only toggle, so it is always written to the user-global config - never the project-local `.ragent/ragent.json`. Project config is untrusted repository content whose `yolo` key is stripped by `Config::merge_project` (SECTASKS T-011), so a project-file write could never round-trip on the next load (and would let a repo turn YOLO on for a reviewer). YOLO state is persisted via `persist_yolo`, which delegates to the runtime-flag helper's `persist_to_global`:

```rust
pub fn persist_yolo(enabled: bool) -> anyhow::Result<()> {
    // RuntimeFlag::persist_to_global: set the single top-level `yolo` key in
    // the raw global file, preserving every other key, then write atomically.
    YOLO_MODE.persist_to_global(enabled)
}
```

The `persist_to_global` path goes through `Config::set_global_bool_key`, which reads the global file as raw JSON, sets the one key, and writes it back through the atomic write helper (temp file + `fsync` + rename). It deliberately does *not* re-serialise the merged `Config`, so the project overlay and any key the running build does not model are never folded into the user's global file.

The `yolo` field in `ragent.json`:

```json
{
  "yolo": false
}
```

On startup, `sync_from_config()` reads this field and sets the runtime flag:

```rust
pub fn sync_from_config() {
    let enabled = crate::config::Config::load()
        .map(|c| c.yolo)
        .unwrap_or_default();
    set_enabled(enabled);
}
```

7.5 Status Bar Indicator

The status bar shows the YOLO state as `YOLO` with a green (enabled) or red (disabled) indicator:

```
YOLO:   (enabled - safety checks bypassed)
YOLO:   (disabled - normal safety checks)
```

7.6 YOLO vs. Autopilot

Feature	YOLO Mode	Autopilot
Bypasses bash safety	Yes	No
Auto-approves perms	Yes (all)	No (respects rules)
Auto-continues turns	No	Yes
Persists to config	Yes	No (session only)
Time/token limits	No	Yes

For fully unattended operation, enable both YOLO and autopilot. For autonomous operation with safety, use autopilot alone with a permissive ruleset.

---

8. The Interactive Permission Dialog

When the permission system returns **Ask**, the agent publishes a `PermissionRequested` event on the event bus. The TUI renders a modal dialog and waits for the user to respond.

8.1 Dialog Layout

```
    Permission Required (1:45 remaining)

  Permission: file:write

  Details:
  write: src/main.rs

  Press [y] to allow  [a] to always allow  [n] to deny
```

The dialog is centered, 60% width and 40% height, with a double border and yellow-on-black styling for emphasis.

8.2 Countdown Timer

The dialog displays a live countdown timer (format `M:SS`) that decrements without requiring keyboard input. The event loop redraws continuously while the dialog is visible.

- Default timeout: **120 seconds** (2 minutes)
- When the timeout is reached, the dialog shows `(EXPIRED)` and the request is auto-denied

The timeout is implemented in `check_permission_with_prompt`. The prompt window defaults to **120 seconds**; on timeout the request is auto-denied. Loop runs can override the window per run via `--timeout`

N, which flows into checkpoint_timeout_secs (Option<u32> on the LoopSpec; None falls back to the loop.checkpoint_timeout_secs config value, default 120):

```
// Reply window: loop checkpoint override, else the 120-second default.
let reply_window_secs = if checkpoint_forced {
    checkpoint_timeout_secs
} else {
    120
};
let deadline = tokio::time::Instant::now()
    + tokio::time::Duration::from_secs(u64::from(reply_window_secs));

loop {
    let recv_timeout = deadline.saturating_duration_since(tokio::time::Instant::now());
    if recv_timeout.is_zero() {
        debug!("Permission request timeout for {tool_name}");
        return Ok(PermissionAction::Deny);
    }
    // ... wait for reply
}
```

8.3 Keyboard Controls

Key	Action
y	Allow this single occurrence (decision = Once)
a	Allow now and for all future matching requests (decision = Always)
n	Deny the request (decision = Deny)

When the user presses a, the checker records an always-grant via record_always:

```
if allowed && decision == PermissionDecision::Always {
    let mut c = checker.write();
    c.record_always(permission, resource);
}
```

This grant persists for the lifetime of the session and takes precedence over all ruleset entries.

8.4 Queue

If multiple permission requests arrive while one is pending, they are queued in permission_queue (a VecDeque). The dialog shows the queue depth:

```
Permission Required (1:45 remaining) (3 queued)
```

Requests are displayed one at a time as earlier ones are resolved.

8.5 Permission Decision Types

The PermissionDecision enum (defined in crates/ragent-types/src/permission.rs) captures the user's response:

```rust
pub enum PermissionDecision {
    Once,    // Allow this single occurrence only
    Always,  // Allow now and for all future matching requests
    Deny,    // Deny the request
}
```

---

9. HTTP Server Permission Flow

When running ragent as an HTTP server (`ragent serve`), permission requests are delivered as SSE events and replies are sent via a REST endpoint.

9.1 SSE Event

When a tool needs permission, the server emits a `permission_requested` SSE event:

```json
{
  "type": "permission_requested",
  "data": {
    "session_id": "abc-123",
    "request_id": "uuid-456",
    "permission": "file:write",
    "description": "write: src/main.rs",
    "options": []
  }
}
```

9.2 REST Reply

The frontend replies by POSTing to the permission endpoint:

```
POST /sessions/{id}/permission/{req_id}
Content-Type: application/json

{ "decision": "allow" }
```

Valid decisions:

Decision	Effect
`allow`	Allow this single occurrence (decision = Once)
`always`	Allow now and for all future matching requests
`deny`	Deny the request

The server maps these to `PermissionDecision` values and publishes a `PermissionReplied` event on the event bus, which unblocks the waiting `check_permission_with_prompt` call.

See `docs/howtos/teams.md` for how team coordination works over the API, and the project root `SPEC.md` Section 7 for the full HTTP API reference.

---

10. Hardwired Auto-Approve Rules

Some tools are **always auto-approved** and never trigger an interactive prompt, regardless of the configured ruleset. This is because they are read-only helpers that pose no risk to the system.

Defined in `crates/ragent-agent/src/session/permissions.rs`:

```rust
pub(crate) fn is_hardwired_auto_approved_tool(tool_name: &str) -> bool {
    is_hardwired_codeindex_tool(tool_name)
        || tool_name.starts_with("team_")
        || AUTO_APPROVED_AGENT_TOOLS.contains(&tool_name)
        || TASK_TOOLS.contains(&tool_name)
        || tool_name == "AskUserQuestion"
        || tool_name == "model_info"
        || tool_name == "ragent_info"
        || tool_name == "tool_info"
        || tool_name == "commands_info"
}

pub(crate) fn is_hardwired_codeindex_tool(tool_name: &str) -> bool {
    const AUTO_APPROVED_CODEINDEX_TOOLS: &[&str] = &[
        "codeindex_search",
        "codeindex_symbols",
        "codeindex_references",
        "codeindex_dependencies",
        "codeindex_status",
        "codeindex_reindex",
    ];
    AUTO_APPROVED_CODEINDEX_TOOLS.contains(&tool_name)
}

const TASK_TOOLS: &[&str] = &["TaskCreate", "TaskGet", "TaskList", "TaskUpdate"];

const AUTO_APPROVED_AGENT_TOOLS: &[&str] = &[
    "new_agent",
    "cancel_agent",
    "list_agents",
    "wait_agents",
    "agent_complete",
];
```

The four `Task*` tools are an explicit list rather than a `task_` prefix test: a prefix admits any future name starting with those five characters, and it named a spelling the rename retired.

Tool family	Tools
Codeindex (read-only)	`codeindex_search`, `codeindex_symbols`, `codeindex_references`, `codeindex_dependencies`, `codeindex_status`, `codeindex_reindex`
Sub-agent management	`new_agent`, `cancel_agent`, `list_agents`, `wait_agents`, `agent_complete`

Tool family	Tools
Team tools	All tools starting with `team_` (19 tools)
Task management	`TaskCreate`, `TaskUpdate`, `TaskGet`, `TaskList`
Interactive question	`AskUserQuestion`
Model introspection	`model_info`

> **Note:** the graph tools (`codeindex_explain`, `codeindex_path`, `codeindex_communities`, `codeindex_godnodes`) are **not** in the hardwired codeindex list above; they follow the normal permission path.

These tools are checked **first** in `check_permission_with_prompt`, before any other layer. See `docs/howtos/codeindex.md` for details on the code index tools and `docs/howtos/teams.md` for the team coordination tools.

---

11. Configuration Reference

11.1 Configuration Sources

ragent loads configuration with the following precedence (highest first):

1. `--config <PATH>` CLI argument
2. `RAGENT_CONFIG_CONTENT` environment variable (inline JSON)
3. `RAGENT_CONFIG` environment variable (path to config file)
4. `.ragent/ragent.json` (or `ragent.jsonc`) in the working directory
5. `~/.config/ragent/ragent.json` (global config)
6. Built-in defaults

Global and project configs are **merged** — the union of all entries is used. For arrays like `permissions`, `bash.allowlist`, and `dirs.allowlist`, entries from all sources are combined.

11.2 Permission-Related Config Fields

```
{
  // Global permission rules applied to all agents
  "permissions": [
    { "permission": "edit", "pattern": "src/**", "action": "allow" },
    { "permission": "bash", "pattern": "cargo *", "action": "allow" },
    { "permission": "edit", "pattern": "secrets/**", "action": "deny" }
  ],

  // Bash command allow/deny lists (Layers 2 and 6)
  "bash": {
    "allowlist": ["curl", "kubectl"],
    "denylist": ["docker rm", "terraform destroy"]
  },

  // Directory/file path allow/deny lists
  "dirs": {
    "allowlist": ["src/**/*.rs", "docs/**/*.md"],
```

```
    "denylist": ["secrets/**", "*.env"]
  },

  // YOLO mode - bypass all command validation and tool restrictions
  "yolo": false
}
```

> **Note:** every registered tool is advertised to the model. Permissions, not advertising, control whether a tool call is **allowed** to execute; see the rule sections above.

11.3 CLI Flags

Flag	Alias	Effect
`--yes`	`--no-prompt`	Auto-approve all permissions (session only)
`--config PATH`		Override config file path
`--no-tui`		Run without TUI (plain stdout)

The `--yes` flag sets `auto_approve = true` on the session processor, which causes `check_permission_with_prompt` to return `Allow` immediately without checking rules or prompting:

```rust
if auto_approve {
    return Ok(PermissionAction::Allow);
}
```

---

12. Custom Agent Permission Rules

Custom agents (OASF `.json` or `.md` profiles) can define their own `permissions` array, which **replaces** the default ruleset for that agent. See `docs/howtos/custom-agents.md` for the full custom agent schema.

12.1 OASF JSON Example

```json
{
  "modules": [{
    "type": "ragent/agent/v1",
    "payload": {
      "system_prompt": "You are a technical writer...",
      "permissions": [
        { "permission": "read",  "pattern": "**",      "action": "allow" },
        { "permission": "edit",  "pattern": "docs/**", "action": "allow" },
        { "permission": "edit",  "pattern": "**/*.md", "action": "allow" },
        { "permission": "edit",  "pattern": "**",      "action": "ask"   },
        { "permission": "bash",  "pattern": "**",      "action": "deny"  }
      ]
    }
  }]
}
```

12.2 Markdown Profile Example

```yaml
---
name: security-reviewer
permissions:
  - { permission: read, pattern: "**", action: allow }
  - { permission: edit, pattern: "**", action: deny }
  - { permission: bash, pattern: "**", action: deny }
---

You are a security-focused code reviewer...
```

12.3 Common Patterns

Read-only reviewer:

```jsonc
"permissions": [
  { "permission": "read", "pattern": "**", "action": "allow" },
  { "permission": "edit", "pattern": "**", "action": "deny" },
  { "permission": "bash", "pattern": "**", "action": "deny" }
]
```

Docs-only writer:

```jsonc
"permissions": [
  { "permission": "read", "pattern": "**",      "action": "allow" },
  { "permission": "edit", "pattern": "docs/**", "action": "allow" },
  { "permission": "edit", "pattern": "**/*.md", "action": "allow" },
  { "permission": "edit", "pattern": "**",      "action": "ask"   },
  { "permission": "bash", "pattern": "**",      "action": "deny"  }
]
```

Full-access coder (with confirmation):

```jsonc
"permissions": [
  { "permission": "read", "pattern": "**", "action": "allow" },
  { "permission": "edit", "pattern": "**", "action": "ask"   },
  { "permission": "bash", "pattern": "**", "action": "ask"   }
]
```

---

13. Slash Commands

13.1 `/bash` — Bash Command List Management

```
/bash show                          Show all command lists
/bash add allow <cmd>               Allow a banned command prefix
/bash add deny <pattern>            Block any command containing <pattern>
/bash remove allow <cmd>            Remove from allowlist
/bash remove deny <pattern>         Remove from denylist
/bash help                          Show help
```

```
# Append --global to write to ~/.config/ragent/ragent.json
# instead of .ragent/ragent.json
```

13.2 `/dirs` — Directory/File Permission Management

```
/dirs show                              Show allowlist and denylist
/dirs add allow <pattern>               Add a glob pattern to auto-allow
/dirs add deny <pattern>                Add a glob pattern to auto-deny
/dirs remove allow <pattern>            Remove from allowlist
/dirs remove deny <pattern>             Remove from denylist
/dirs help                              Show help

# Append --global to write to global config
```

Pattern matching uses glob syntax: - `*` matches any sequence of characters (except `/`) - `**` matches any sequence of characters (including `/`) - `?` matches any single character - `[abc]` matches any character in the set

13.3 `/yolo` — Toggle YOLO Mode

```
/yolo                                   Toggle YOLO mode on/off
```

Also toggled with `Alt+Y`. Persists to `ragent.json`.

13.4 `/autopilot` — Autonomous Operation

```
/autopilot on                           Enable autopilot
/autopilot on --max-tokens N            Enable with token budget
/autopilot on --max-time N              Enable with time budget (seconds)
/autopilot off                          Disable autopilot
/autopilot status                       Show current state
```

---

14. Worked Examples

14.1 Safe Development Setup

A typical project config that allows the agent to work freely within `src/` but prompts for anything else:

```
{
  "permissions": [
    { "permission": "read", "pattern": "**", "action": "allow" },
    { "permission": "edit", "pattern": "src/**", "action": "allow" },
    { "permission": "edit", "pattern": "tests/**", "action": "allow" },
    { "permission": "edit", "pattern": "docs/**", "action": "allow" },
    { "permission": "edit", "pattern": "**", "action": "ask" },
    { "permission": "bash", "pattern": "cargo *", "action": "allow" },
    { "permission": "bash", "pattern": "git *", "action": "allow" },
    { "permission": "bash", "pattern": "**", "action": "ask" }
  ],
  "dirs": {
```

```
    "denylist": ["secrets/**", "*.env", ".ssh/**", ".aws/**"]
  }
}
```

14.2 CI/CD Agent with Network Access

An agent that can fetch from the web and push to git, but cannot modify files outside the build output:

```
{
  "permissions": [
    { "permission": "read", "pattern": "**", "action": "allow" },
    { "permission": "edit", "pattern": "target/**", "action": "allow" },
    { "permission": "edit", "pattern": "**", "action": "deny" },
    { "permission": "web", "pattern": "**", "action": "allow" },
    { "permission": "bash", "pattern": "git push *", "action": "allow" },
    { "permission": "bash", "pattern": "cargo *", "action": "allow" },
    { "permission": "bash", "pattern": "**", "action": "deny" }
  ],
  "bash": {
    "allowlist": ["curl"]
  }
}
```

14.3 Read-Only Code Reviewer

A locked-down agent that can only read and search, with no write or shell access:

```
{
  "permissions": [
    { "permission": "read", "pattern": "**", "action": "allow" },
    { "permission": "edit", "pattern": "**", "action": "deny" },
    { "permission": "bash", "pattern": "**", "action": "deny" },
    { "permission": "web", "pattern": "**", "action": "deny" }
  ]
}
```

14.4 Fully Autonomous (YOLO + Autopilot)

For trusted local environments where you want the agent to work completely unattended:

```
# Start with YOLO mode and auto-approve
ragent --yes

# In the TUI, enable autopilot with limits
/autopilot on --max-tokens 200000 --max-time 3600
```

Or configure in `ragent.json`:

```
{
  "yolo": true
}
```

Then in the TUI:

```
/yolo
/autopilot on --max-time 1800
```

> **Warning:** YOLO mode bypasses all safety checks. Only use it in disposable environments (containers, VMs) or when you fully trust the agent and its inputs.

14.5 Selective Bash Access

Allow specific commands without prompting while keeping everything else interactive:

```
{
  "bash": {
    "allowlist": ["curl", "kubectl", "terraform"],
    "denylist": ["docker rm -f", "kubectl delete"]
  },
  "permissions": [
    { "permission": "bash", "pattern": "cargo *", "action": "allow" },
    { "permission": "bash", "pattern": "git status", "action": "allow" },
    { "permission": "bash", "pattern": "git diff *", "action": "allow" },
    { "permission": "bash", "pattern": "git log *", "action": "allow" },
    { "permission": "bash", "pattern": "**", "action": "ask" }
  ]
}
```

14.6 Denying Access to Secrets

Block access to credential files regardless of other rules:

```
{
  "dirs": {
    "denylist": [
      "secrets/**",
      "*.env",
      ".env*",
      "**/.aws/**",
      "**/.ssh/**",
      "**/credentials*",
      "**/*.pem",
      "**/*.key"
    ]
  }
}
```

The denylist is checked before the permission ruleset, so these patterns override any `allow` rules.

See Also

Document	Topic
`docs/howtos/tutorial.md`	TUI walkthrough, keybindings, getting started

Document	Topic
`docs/howtos/tools/INDEX.md`	Complete tool catalog with schemas (by category)
`docs/howtos/custom-agents.md`	Custom agent definitions with permission rules
`docs/howtos/hooks.md`	PreToolUse/PostToolUse lifecycle hooks
`docs/howtos/llmsecurity.md`	LLM security analyzer (tightening-only layer)
`docs/howtos/codeindex.md`	Code index tools (hardwired auto-approve)
`docs/howtos/teams.md`	Team coordination tools
`SPEC.md` (project root)	Architecture spec, Section 4: Security & Permissions
`QUICKSTART.md` (project root)	Quick start guide